

Physical Intelligence $\pi_{0.5}$ — Unified Engine v1

Apex Compute

1 $\pi_{0.5}$

$\pi_{0.5}$ is a vision–language–action policy: camera views and an instruction in, a chunk of future control steps out. Three stages run in sequence: a SigLIP vision encoder, a Gemma-2B prefix that produces a K/V cache, and a Gemma-300M action expert that denoises the action chunk against that cache.

The prefix is three quarters of the arithmetic (Table ??) and bounds compute. The expert is small but strictly sequential: ten Euler steps of $x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, \text{KV}_{\text{prefix}})$, so it bounds latency and its error compounds across steps.

Table 1: Per-inference budget at the model’s true shape.

Stage	Shape	Params	GFLOP
Vision (SigLIP ViT)	27 L, $d1152$, 3×256 tok	~ 0.41 B	655.9
Prefix (Gemma-2B)	18 L, $d2048$, 576 tok	~ 2.0 B	2332.0
Expert (Gemma-300M)	18 L, $d1024$, 10 steps	~ 0.31 B	71.0
Total		~ 2.7 B	3058.9

2 Porting

The Unified Engine v1 is a soft accelerator IP, instanced twelve times on one AMD Alveo U55C card with 4 GB of DRAM. Two constraints shape the port: the DRAM budget and the engine’s 64-element alignment.

Weights. The bf16 checkpoint does not fit, so every matmul weight is 4-bit block-quantized ($\sim 8 \times$ smaller) and dequantized to bf16 at the MAC. IF4 buys footprint, not speed.

Vision head dim. Head dim 72 is not 64-aligned. Attention runs at 128 and is contracted back through a precomputed selection matrix; the $Q/K/V/O$ projections stay at 72.

Prefix masking. A masked image slot is dropped from prefill, a quarter less work with bit-identical output. Rotary positions are cumulative-sum-of-mask indices, not **arange**, so removing a slot does not shift phase.

Denoise loop. All ten steps compile into one program and one execute call. Every strided SRAM→DRAM copy needs an explicit destination offset; a missing offset scrambles the output silently, with no NaN.

3 Quantization

3.1 IF4

IF4 is the accelerator’s 4-bit weight format. Each 64-element block is quantized as signed INT4 and as FP4 E2M1, and the lower-error encoding is kept.

The choice is carried by the sign bit of the block’s bf16 scale, which is an absmax quotient and therefore never negative. No side table, no extra bytes. Decode is a 2:1 mux ahead of the MAC over the INT4 and FP4 paths that already exist. The per-block search runs once, offline.

Why a hybrid. INT4 spends its 16 levels uniformly; FP4 spends them densely near zero and sparsely at the tails, with a wide gap below its largest code. Which one wins is set by the block’s crest factor, the ratio of its peak to its bulk. Dense blocks fill every uniform level; a single outlier stretches the shared scale and starves the rest.

Table ?? shows the crossover. The extremes go as expected. The Gaussian row is the informative one: INT4 and FP4 tie, yet the hybrid still gains, because crest factor varies block to block within a single tensor. The INT4 share rises with depth through the vision tower and is lowest in the token embedding, the most heavy-tailed weight in the model.

Table 2: Weight-domain SNR (dB): INT4, FP4, and the IF4 hybrid, with the fraction of blocks the hybrid assigns to INT4. Synthetic blocks above, $\pi_{0.5}$ checkpoint group means below.

Block source	INT4	FP4	IF4	% $\rightarrow$ INT4
Uniform (bounded)	23.12	19.36	23.12	99.9
Gaussian	19.35	19.52	20.11	49.5
Gaussian + one outlier	11.81	16.50	16.50	0.0
Action expert	19.50	19.47	20.25	54.6
Prefix LM	19.05	19.47	20.00	45.2
Vision encoder	18.31	19.23	19.65	37.4
Token embedding	15.39	18.53	18.65	17.6

Scope: the build measured below pins the INT4 branch. The hybrid’s gain is a weight-domain result and has not yet been carried end-to-end.

3.2 The quantization floor

Table ?? is the quantizer’s cost in isolation: the same torch run, inputs and noise at IF4 and at bf16, diffed per stage, with no hardware in the loop.

Vision is nearly untouched. The prefix caches degrade with depth, V more than K. The velocity field degrades over the last Euler steps as the integrator accumulates error, and the final action lands at 28.5 dB. The loss sits in the low-magnitude action dimensions; the gripper holds above 50 dB. SNR is a diagnostic; the gate is closed-loop success (Sec. ??).

Table 3: IF4 vs. bf16 per section (torch reference, sample 0).

Model section	SNR (dB)	Range (dB)
Vision encoder (3 cameras)	25.39	24.3–26.6
Prefix K-cache (18 layers)	16.57	13.6–19.9
Prefix V-cache (18 layers)	11.40	8.1–17.0
Denoise velocity v_t (10 steps)	31.51	22.2–34.5
End-to-end action (10$\times$7)	28.47	MSE 2.22×10^{-4}

3.3 LIBERO

LIBERO is a tabletop manipulation benchmark for language-conditioned policies, scored by closed-loop task success. It provides two camera views; $\pi_{0.5}$ takes three, so the third slot is zero-filled and masked. Encoding it costs nothing in correctness and keeps the three-camera path exercised.

An episode is many inferences. The simulator executes the first ten steps of each chunk and requests the next, so one episode is a dozen or more full passes on hardware. Table ?? compares the FPGA policy with the CUDA reference over ten prompts at a fixed seed and fixed denoise noise, so both see byte-identical observations.

Table 4: LIBERO closed-loop outcomes, FPGA (IF4) vs. CUDA (bf16): 10 prompts, one episode each, fixed seed and noise. ✓ success, × failure, number = episode length in sim steps. Δ = FPGA – CUDA steps on successes; failures run to the cap.

Prompt	FPGA (IF4)	CUDA (bf16)	Δ steps	Agree
0	✓ 94	✓ 95	−1	✓
1	✓ 160	✓ 126	+34	✓
2	✓ 123	✓ 115	+8	✓
3	✓ 108	✓ 109	−1	✓
4	× 230	× 230	—	✓
5 [†]	× 290	× 290	—	✓
6	✓ 121	✓ 120	+1	✓
7	× 230	× 230	—	✓
8	✓ 107	✓ 106	+1	✓
9	× 230	× 230	—	✓
Rate	6/10	6/10		10/10

[†]Prompt 5 is a `libero_object` task; the original `libero_spatial` task 5 exercised a denoise-loop compile path that crashed the FPGA bring-up, so it is substituted with the next unused prompt.

The two policies agree on every prompt, and successful episodes differ by a few steps.

4 Throughput: Alveo 12-engine vs. CUDA

Table ?? lists the three platforms and their power. Each is then reported as per-section wall-clock: Alveo 12-engine times come from the engine’s own counters; the GPU baseline is openpi’s JAX/XLA path.

Table 5: Platforms. Power is the board draw during inference.

Platform	Precision	Runtime	Power
Alveo 12-engine (AMD Alveo U55C, 12×Unified Engine v1)	IF4 weights, bf16 MAC	this work	— W
RTX 5070	bf16	openpi JAX/XLA	— W
Jetson Orin Nano Super	bf16	openpi PyTorch/CUDA	7 W mod

GPU stages are jitted separately with a device sync at each boundary, so their sum slightly exceeds the fused end-to-end time (Table ??).

Table 6: RTX 5070 (bf16, openpi JAX/XLA). Per-stage times are separately-jitted; End-to-end is the fused `policy.infer`.

Model section	Time (ms)	GFLOP/s
Vision encoder	19.41	33 793
Prefix (Gemma-2B)	79.29	29 411
Action expert (denoise)	36.70	1 935
End-to-end	130.15	23 502
Throughput (inf/s)	7.68	

Edge GPU. The Jetson Orin Nano Super runs the same checkpoint at bf16 in 7 W mode (Table ??).

Table 7: Jetson Orin Nano Super (bf16, openpi PyTorch/CUDA), 968-token prefix, 7 W mode.*

Model section	Time (ms)	GFLOP/s
Vision encoder	480.1	1 366
Prefix (Gemma-2B)	2 357.8	1 686
Action expert (denoise)	1 427.6	53.7
Weight streaming*	5 088.6	—
Compute only	4 265.5	1 104
End-to-end	9 354.1	503
Throughput (inf/s)	0.107	

*The checkpoint exceeds the Jetson’s usable memory, so sections run staged and the prefix re-reads its layer weights from storage each inference. Not model compute.

Achieved rates fall as sections shrink. The expert is the extreme case: ten short sequential steps against a long K/V cache are latency-bound, and arithmetic throughput does not help.

The Jetson prefix is longer than the Alveo 12-engine’s because openpi’s PyTorch path encodes all three slots and pads the instruction to its full budget, so per-section GFLOP counts differ between the GPU and Jetson columns.

Table 8: Alveo 12-engine (IF4) — twelve Unified Engine v1 instances on one AMD Alveo U55C, 832-token prefix. Times are pure back-to-back execution; programs are compiled once up front and that one-time cost is excluded.

Model section	Time (s)
Vision encoder	2.6
Prefix (Gemma-2B)	8.6
Action expert (denoise)	0.9
End-to-end	12.1
Throughput (inf/s)	0.0826

The expert is the least efficient stage here for the same reason as on the Jetson: it is a chain of short sequential steps, not a throughput problem.

Config. Alveo 12-engine: 4 GB DRAM. All platforms batch 1, ten denoise steps, sample 0.

The whole $\pi_{0.5}$ policy runs on the Unified Engine v1, vision, prefix and the full denoise loop, with no host fallback (Table ??). It is two orders of magnitude behind the desktop GPU and within a small factor of the Jetson. FLOPs concentrate in the prefix; latency concentrates in the expert.

And IF4 pays for its footprint only in action dimensions the policy does not act on.